

JavaScript - Fundamentals

Contents

1. Introduction, Concepts, Features, Version
2. Variable declaration, Operators, Control Statements, DOM
3. Events
4. Functions – Normal, Anonymous and Arrow Functions
5. Hoisting
6. Exception handling
7. JS Architecture
8. Automation Software
9. JS for Automation

Introduction

- ✓ **JavaScript** was created by **Brendan Eich** at **Netscape** in **1995**.
- ✓ **Netscape** submitted **JavaScript** to **ECMA** for standardization because without standardization problems would arise with **compatibility across browsers**.
- ✓ Soon after, different browsers started implementing JavaScript differently.
- ✓ Analogy - ECMAScript = Specification (rules)
JavaScript = Implementation

Concepts

- ✓ **HTML** is a free – form language providing only static content.
- ✓ **Java Script** is a **SCRIPTING LANGUAGE** embedded into **HTML page** for the purpose of making it **dynamic** e.g. objects, events, implicit timers, etc.
- ✓ **JavaScript** is **case sensitive i.e.** variable names are case sensitive, but not strongly-typed instead its very loosely-typed language .
- ✓ **JavaScript** though not Java, brings Java features into HTML viz..
variables, program statements, control statements, functions, objects, events.
- ✓ **JavaScript testing** is more important as it is loosely-typed language. It is **much easier to make mistakes** by using objects in ways that weren't intended or just fat-fingering a variable name and user won't find out about it until code is run and error is seen. (a good IDE can also help).

JavaScript features

- ✓ JavaScript is purely **interpreted language** i.e. executed without preliminary compilation.
- ✓ It can
 - ❑ be used for programming on web-pages
 - ❑ react to events
 - ❑ validate user input

```
<html>
```

```
<head></head>
```

```
<body>
```

```
<script>
```

```
document.write ("Hello World!")
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript versions

Version	Official Name	Release Year	Impact & Core Features
ES1	ECMAScript 1	1997	First standardized version of JavaScript.
ES5	ECMAScript 5	2009	Added strict mode, native JSON parsing, and Array methods like .map() and .filter().
ES6	ECMAScript 2015	2015	JavaScript Revolution - milestone. Introduced let/const, arrow functions, classes, and Promises.
ES14	ECMAScript 2023	2023	Introduced immutable array methods like .toReversed() and .findLast().
ES15	ECMAScript 2024	2024	Introduced Map grouping features (Object.groupBy) and atomic enhancements
ES16	ECMAScript 2025	2025	Refined modern synchronization tools, Set methods, and module improvements

Variable declaration

- ✓ JavaScript variables are **loosely typed**, meaning they have no datatype. Only datatype is '**var**'.
- ✓ JavaScript provides three ways / keywords to declare a variable – **var**, **const**, and **let**. **var** is a traditional method to declare variable. **const** and **let** keywords were introduced later by the **ES2015/ES6**.
- ✓ Traditionally there were only **two types** of **variable scopes** in JavaScript: **Global** and **Function**.
- ✓ **let** and **const** create **block** scope variable.
- ✓ These keywords deliver different functionality from each other.
- ✓ **document.write** command is a standard JavaScript command for writing output to a page. The statement **document.write** is entered the between the tags **<script>** **</script>**.
- ✓ Browser recognizes it as a JavaScript command and execute the code line.

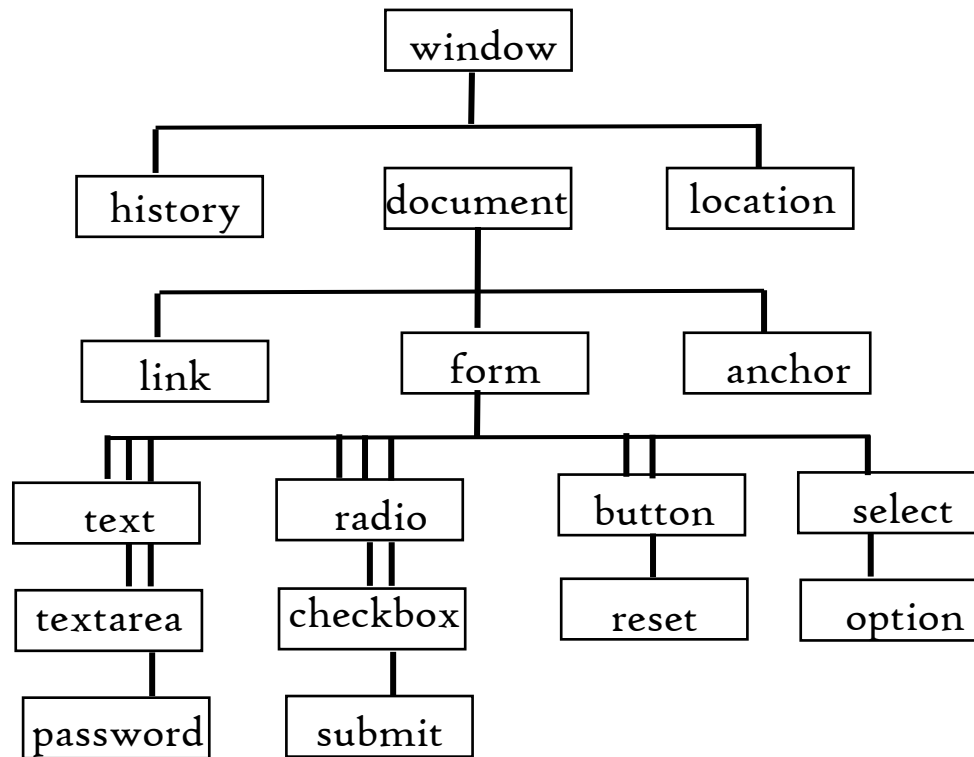
Variable scope

- ✓ While `const` is fundamentally block-scoped, its behavior changes depending on where you declare it:
 - a. **Block Scope:** If declared inside an `if` statement, `for` loop, or a standalone block `{}`, it cannot be accessed outside of that block.
 - b. **Function Scope:** If declared inside a function, it is scoped locally to that function and cannot be accessed outside.
 - c. **Global Scope:** If declared at the top level (outside any function or block), it is globally accessible. However, unlike `var`, top-level `const` variables **do not** become properties of the global window or global `this` object.

Operators in JavaScript

Sl.No.	Category	Operators
1.	Arithmetic	+, -, *, /, %, ++, --
2.	Assignment	=, +=, -=, *=, /=, %=
3.	Relational	==, ===, <, >, !=, >=, <=
4.	Logical	&&,
5.	Conditional	? :
6.	String	+

Document Object Model of JS



JS Conditional Statements

✓ Conditional Statements –

- ❑ if
- ❑ if...else
- ❑ else if ladder
- ❑ nested if

✓ Switch statement –

- ❑ switch(var)

JS Loop statements

➤ Repetitive Statements

- ❑ while(test-condition) { }
- ❑ do {
 }
 } while(test-condition)
- ❑ for(initial; test-condition; modifier)
 { }
- ❑ for (var in collection) { }
- ❑ for(var of iterable) { }
- ❑ key words in repetitive statements –
 - ✓ break
 - ✓ continue

Events

- ✓ **Events** are actions that can be detected by JavaScript
- ✓ **Event-driven** systems **communicate** through events – messages that signal that something has happened. **Event-based communication** enables flexibility, performance, and maintainability across entire stack – browser interactions to backend microservices.
- ✓ **Event** can have any number of event handlers (event handlers are pieces of JS code that execute when that event occurs).
- ✓ Event handlers' names usually start with “on”.

contd..

Event	Handler	Description
click	onclick	when mouse click on an element
mouseover	onmouseover	when the cursor of the mouse comes over the element
mouseout	onmouseout	when the cursor of the mouse leaves an element
mousedown	onmousedown	when the mouse button is pressed over the element
mouseup	onmouseup	when the mouse button is released over the element
mousemove	onmousemove	when the mouse movement takes place.

contd..

Event	Handler	Description
focus	onfocus	when the user focuses on an element
submit	onsubmit	when the user submits the form
blur	onblur	when the focus is away from a form element
change	onchange	when the user modifies or changes the value of a form element
keydown and keyup	onkeydown & onkeyup	when the user press and then release the key

contd..

Event	Handler	Description
load	onload	when the browser finishes the loading of the page
unload	onunload	when the visitor leaves the current webpage, the browser unloads it
resize	onresize	when the visitor resizes the window of the browser

Functions

✓ Functions

- ❑ Used mainly for code reuse
- ❑ Generally defined in the <head> and used in the <body>
- ❑ May or may not have arguments
- ❑ May or may not return a value

Anonymous functions and Arrow functions

✓ Anonymous Functions – is a function **without a name**. Since it has no name, it is usually **assigned to variable** or **passed** as an **argument**.

✓ **Syntax**

```
function() {  
    // code  
}
```

✓ **Use Anonymous Functions in scenarios –**

- ❑ One-time use functions
- ❑ Callback functions
- ❑ Event handlers
- ❑ Reduces unnecessary function names

contd..

- ✓ Arrow Functions – a shorter way to write functions. Arrow functions were introduced in ES6 (ECMAScript 2015). Since there is only one statement, return is implied.
- ❑ In a normal function, this refers to the object that calls the function.
- ❑ In an arrow function, this is inherited from the surrounding scope.
- ❑

```
function add(a, b) {  
  return a + b;  
}
```
- ❑

```
const add = (a, b) => {  
  return a + b;  
};
```
- ❑

```
let display = function(name) {  
  console.log(name);  
};
```
- ❑

```
let display = (name) => { console.log(name); };
```

contd..

```
❑ let employee = {  
    name: "John",  
  
    show: () => {  
        console.log(this.name);  
    }  
};  
employee.show();
```

❑ **Output** – undefined

❑ Reason: Arrow functions do not create their own this.

❑ Therefore, for **object methods**, use regular functions.

contd..

Feature	Anonymous Function	Arrow Function
Has name	No	Usually assigned to variable
Introduced in	Original JavaScript	ES6
Syntax	Longer	Shorter
Own this	Yes	No
Suitable as object method	Yes	Usually No
Common use	Callbacks, events	Callbacks, array methods, async code

Getting started with JS programming

Step 1:

```
D:\scanit_javascript> npm init -y
```

Wrote to

```
D:\scanit_javascript\package.json:
```

```
{  
  "name": "scanit_javascript",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\"  
&& exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

Step 2:

```
D:\scanit_javascript> npm install
```

```
up to date, audited 1 package in  
591ms
```

```
found 0 vulnerabilities
```

```
PS D:\scanit_javascript>
```

Hoisting

- ✓ Hoisting is JavaScript's behavior of **moving declarations** to the top of their **scope** during **compilation phase** before **code execution**.
- ✓ Hoisting provides for **processing variable** and **function declarations** before executing the code, making them available earlier in their scope.
- ✓ It does **not** move the actual values; only the declarations are processed first.
- ✓ Provides several benefits to the JavaScript engine and developers.

1. Allows Functions to Be Used Before They Are Defined

- ❑ Without hoisting, functions would have to be declared before every use. User can organize code more naturally and place helper functions later in the file.
- ❑ `calculate();`

```
function calculate() {  
  console.log("Calculation done");  
}
```

2. Enables JavaScript's Compilation Phase

Before executing code, JavaScript scans the entire scope and creates memory for variables and functions.

```
var a = 10;
```

```
var b = 20;
```

```
function add() {  
    return a + b;  
}
```

During the memory creation phase, JavaScript already knows:

Variables: a, b

Function: add

Benefit: Faster and more predictable execution because the engine knows what identifiers exist before running the code.

3. Supports Mutual Function Calls

Functions can call each other regardless of their order.

```
first();
```

```
function first() {  
    second();  
}
```

```
function second() {  
    console.log("Second function");  
}
```

Benefit: More flexible program structure.

4. Creates Consistent Scope Handling

JavaScript knows all declarations within a scope before execution begins.

```
function demo() {  
    console.log(x); // undefined  
    var x = 100;  
}
```

The engine already knows that x belongs to the local scope.

Benefit: Prevents accidental lookup in outer scopes.

5. Supports Large Applications

In large projects, developers often organize code logically rather than strictly top-to-bottom.

```
startApplication();
```

```
// hundreds of lines...
```

```
function startApplication() {  
    console.log("Application started");  
}
```

Benefit: Improves readability and maintainability.

✓ Why Hoisting Is Often Considered a Problem

Many bugs came from var hoisting:

```
console.log(count); // undefined
```

```
var count = 10;
```

A beginner might expect an error but gets undefined.

To reduce such issues, modern JavaScript introduced let and const:

```
console.log(count);
```

```
let count = 10;
```

Output:

ReferenceError

This makes mistakes easier to detect.

✓ In Modern JavaScript

The biggest practical benefit of hoisting today is:

- a. Function declarations can be called before their definition.
- b. The JavaScript engine can prepare scope information before execution.
- c. `let` and `const` still use hoisting internally but protect developers through the Temporal Dead Zone (TDZ).
- d. As a best practice, even though hoisting exists, write code as if declarations should appear before use. This makes the code clearer and avoids surprises.

✓ Should we use hoisting?

- ❑ **Variables (var):** No, do not rely on hoisting.
- ❑ **let and const:** Declare before use.
- ❑ **Function declarations:** Using hoisting is acceptable and often improves code organization.
- ❑ **Best practice:** Write code as if declarations should appear before use, except for function declarations where hoisting can be used deliberately.
- ❑ Understand hoisting thoroughly, but rely on it only for function declarations

contd..

- ✓ Example 1: Variable Hoisting with var

```
console.log(name);
```

```
var name = "Vijay";
```

- ✓ JavaScript interprets it approximately as:

```
var name;
```

```
console.log(name); // undefined
```

```
name = "Vijay";
```

Output – undefined

- ✓ The variable declaration is hoisted, but the assignment remains in its original position.

Exceptions

- ✓ An **exception** is an error or unexpected condition that occurs while a JavaScript program is running.
- ❑ If an exception is not handled, the program execution stops and an error message is displayed.
- ❑ **try...catch**
 - a. JavaScript provides try...catch to handle exceptions.
 - b. try...catch...finally**
 - c. The finally block always executes whether an exception occurs or not.
 - d. throw Statement** used to generate custom exceptions.

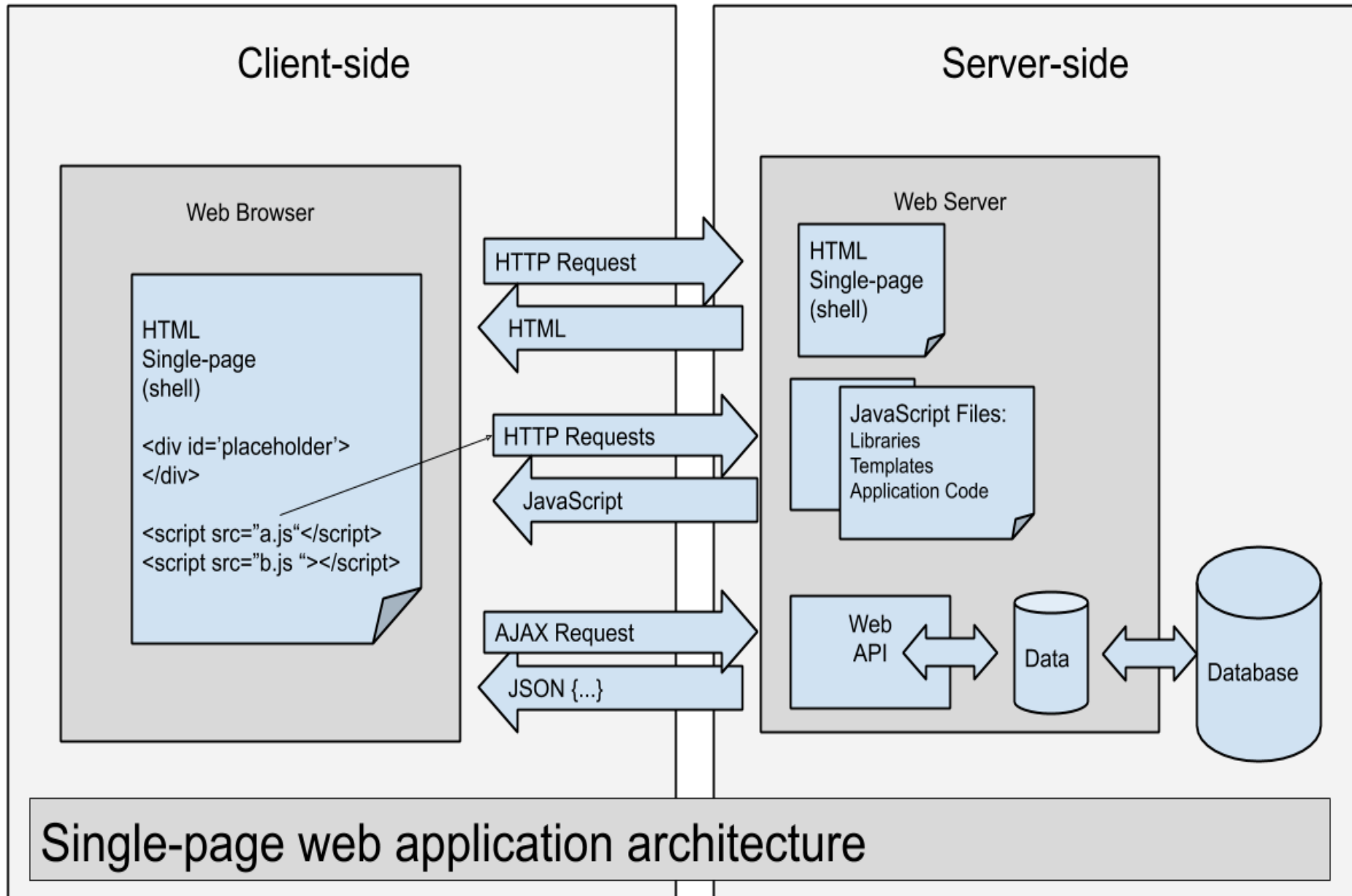
contd..

Error Type	Description
Error	Generic error
TypeError	Wrong data type or invalid operation
ReferenceError	Variable not declared
SyntaxError	Invalid JavaScript syntax
RangeError	Value out of allowed range
URIError	Error in URI encoding/decoding

JavaScript Architecture

- ✓ **Non-blocking** architecture is a **system design** that allows a program to **initiate** a **task** and **continue** executing other operations **without waiting** for that **task** to **finish**. It prevents system threads from sitting idle during slow tasks (like disk reads or network calls), vastly improving an application's scalability and concurrency.
- ✓ **How It Works: Blocking vs. Non-Blocking**
- ✓ **Blocking (Traditional):** When a request is made, the operating system executes the task and forces the execution thread to wait (block) until the task is complete. If 1,000 users make a request, the system often requires 1,000 separate threads, which consumes massive amounts of memory.
- ✓ **Non-Blocking (Asynchronous):** When a request is made, the system registers the task to run in the background and immediately moves on to handle other work. Once the background task is finished, a notification or callback is triggered, returning the result to the main application.

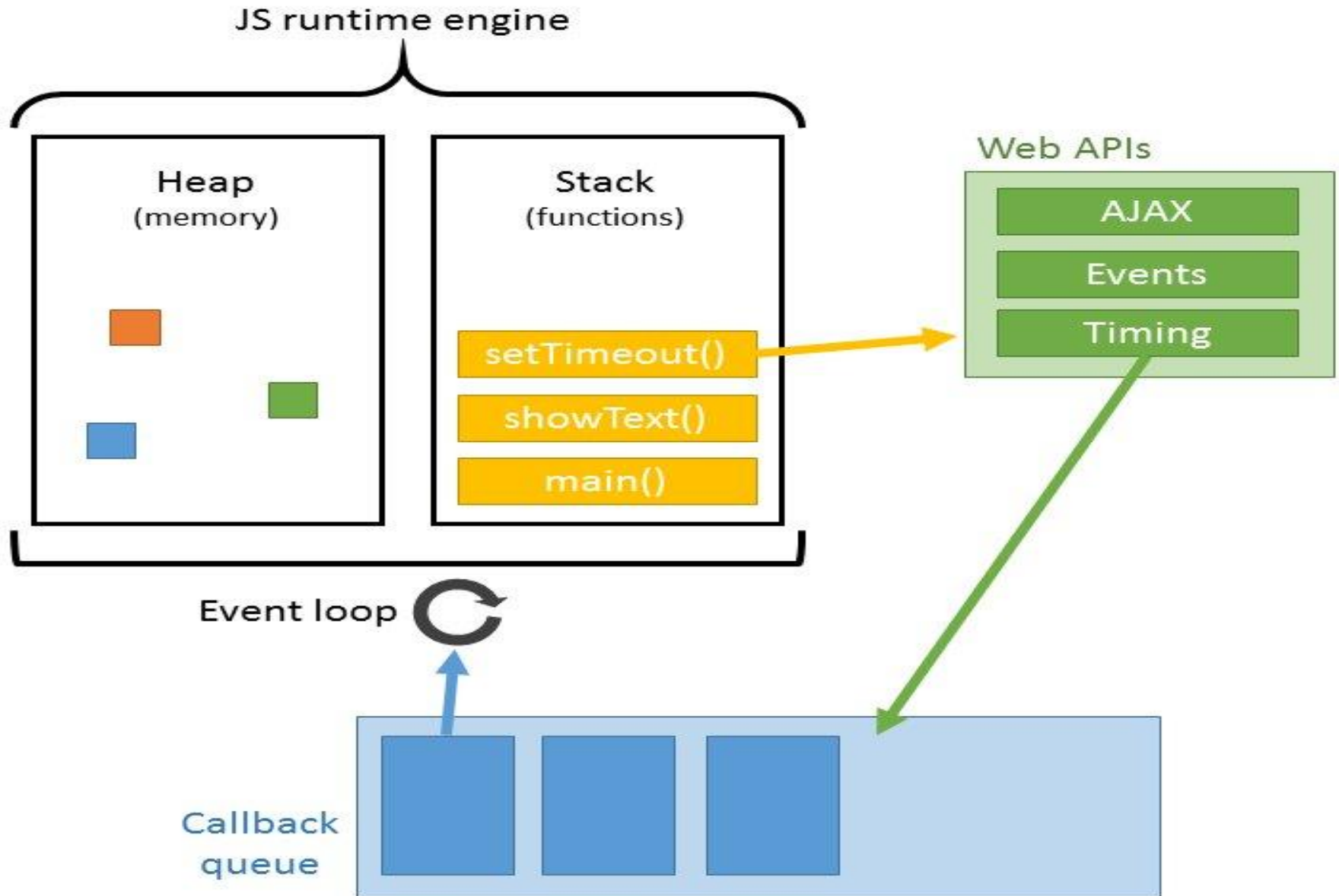
Diagrammatic Representation



Core Components

- ✓ Non-blocking architectures are event-driven and generally rely on a few key mechanics to manage work without causing thread stalls:
- ✓ **The Event Loop:** The central mechanism that continuously monitors for events, executes tasks, and processes completed background callbacks.
- ✓ **Callbacks/Promises:** Functions provided to the system that specify what to do with the data once a background task eventually completes.
- ✓ **Thread Pools:** Subsets of background threads reserved for executing CPU-heavy or specific I/O operations without blocking the main event loop.

contd..



Core Architecture Components

✓JS architecture relies on five primary moving pieces working together.

1. JavaScript Engine: The core interpreter/compiler (e.g., Google's V8, Mozilla's SpiderMonkey). It contains:

a. Memory Heap: Memory allocation happens to variables, objects.

b. Call Stack: Where the engine tracks the execution of function calls (Last-In, First-Out).

c. Environment APIs (Web APIs / Node APIs): Features provided by the environment, not the JS language itself (e.g. DOM, fetch).

2. Callback Queue (Task Queue): A holding area for asynchronous callback functions ready to be executed after their API tasks finish (First-In, First-Out).

3. Microtask Queue: A high-priority queue specifically for Promises and `process.nextTick`. It is completely cleared before the engine moves to the standard Callback Queue.

4. Event Loop: The coordinator. It constantly monitors the Call Stack. If the stack is empty, it pushes the first task from the queues onto the stack to run.

Architectural Workflow Example

- ✓ A practical code snippet illustrating how these pieces interact asynchronously:
- ✓ javascript

```
console.log("1. Start");
setTimeout(
  () => { console.log("2. Timeout Callback (Macro-task)" );
        },
  0);
Promise.resolve().then(() =>
  { console.log("3. Promise Resolved (Micro-task)");
  }
);
console.log("4. End");
```

Execution Step-by-Step

✓ Timeline for internal architecture executes the code block –

Step 1: `console.log("1. Start")` enters the **Call Stack** and executes immediately. *Output:* 1. Start

Step 2: `setTimeout` is pushed to the **Call Stack**. Because it is a Web API function, the JS Engine offloads the timer to the **Browser Environment APIs** and pops `setTimeout` off the stack. The stack is clear while the browser handles the 0ms timer.

Step 3: The timer completes instantly, moving the internal callback function into the **Callback Queue**.

Step 4: The Promise statement hits the **Call Stack**. Its `.then()` resolution is immediately moved into the **Microtask Queue**.

Step 5: `console.log("4. End")` enters the **Call Stack** and runs immediately. *Output:* 4. End

Step 6: The Call Stack is now empty. The **Event Loop** activates and checks the queues. It gives absolute priority to the **Microtask Queue** over the Callback Queue.

contd..

Step 7: The Promise callback is pulled from the Microtask Queue into the Call Stack and runs.

Output: 3. Promise Resolved (Micro-task)

Step 8: The Microtask Queue is completely clear. The **Event Loop** pulls the setTimeout callback from the standard **Callback Queue** into the Call Stack to execute.

Output: 2. Timeout Callback (Macro-task)

Final Scanned Console Output

text

1. Start
4. End
3. Promise Resolved (Micro-task)
2. Timeout Callback (Macro-task)

Real-World Benefits

- ✓ **High Concurrency:** Because threads are not locked waiting for operations to finish, a single server can process thousands of simultaneous connections.
- ✓ **Efficient Resource Usage:** Eliminates the memory and CPU overhead associated with creating and managing massive thread pools.
- ✓ **Responsive APIs:** Ideal for I/O-bound operations (web APIs, microservices) where delays are primarily caused by waiting for data from databases or external networks.
- **Note** – When a HTML file is opened directly from local hard drive, **web browser** acts as the **client**, the computer's **local storage** or **file system** acts as **the server**, and **no network** or **HTTP server** is involved.

Automation software

- ✓ Refers to programs and tools designed to execute digital tasks or business processes with minimal human intervention. By performing rule-based and repetitive actions, it speeds up workflows, reduces human error, and frees up human workers to focus on strategy, creativity, and problem-solving.
- ✓ **Key Types of Automation Software**
 - ❑ **Robotic Process Automation (RPA):** Uses "bots" to mimic human digital actions - copying, pasting data between applications.
 - ❑ **Business Process Automation (BPA):** Streamlines and manages complex, multi-step workflows across an organization e.g. routing a sales lead to the right team and triggering an email.
 - ❑ **IT & Cloud Automation:** Automatically provisions servers, applies software updates (patches), monitors networks, and handles batch processes.
 - ❑ **Test Automation:** Uses scripts to automatically test software applications for bugs, saving QA teams from repetitive manual testing.

- ❑ **Marketing & Sales Automation:** Automatically sends out targeted email campaigns, tracks customer interactions, and manages social media posting schedules.

✓ Common Use Cases

- ❑ **Data Entry:** Transferring customer information from an email directly into a database or CRM.
- ❑ **Customer Support:** Using chatbots to answer common, repetitive customer questions.
- ❑ **Invoice Processing:** Extracting data from incoming bills and sending them to the accounting department for approval.
- ❑ **Notifications & Updates:** Automatically updating inventory levels across various online storefronts when a purchase is made. [[1](#)]

✓ Core Benefits

- ❑ **Speed:** Systems process data in seconds rather than hours.
- ❑ **Accuracy:** Minimizes typos and human errors associated with manual data entry.
- ❑ **Availability:** Bots and automated scripts can run 24/7 without taking breaks.
- ❑ **Cost Efficiency:** Reduces operational costs by optimizing resource allocation and saving employee time.

JS for Automation

- ✓ **JavaScript** is the dominant language for **web automation** because it allows developers and QA teams to use a **single, unified language** across the entire application stack.
- ✓ **JavaScript** is default scripting language for modern websites, using it for automation ensures **flawless alignment** with the **web browser** and **seamless integration** into **engineering teams**.
- ✓ **Why JavaScript Wins in Test Automation**
 1. **Single-Language Ecosystem (Shift-Left Testing)** –
 - Shared Codebases:** Developers and testers use the exact same programming language.
 - Easier Reviews:** Developers can easily review, fix, and maintain automation test scripts.
 - Shift-Left Delivery:** Teams can write automation scripts earlier in the CI/CD DevOps pipeline.

2. Native Browser Integration

Fast Execution: JavaScript runs natively inside web browsers.

Direct DOM Control: Automation scripts can interact with the Document Object Model (DOM) and network layer without slow, flaky intermediary drivers.

3. Powerhouse Modern Frameworks – The evolution of Node.js introduced highly optimized automation engines designed exclusively for modern web architectures:

Playwright: Provides cross-browser support, fast parallel execution, and built-in auto-waiting mechanisms.

Cypress: Executes code right inside the browser for real-time debugging and reliable UI testing.

WebDriverIO: Offers custom plugins and highly scalable web and mobile test execution.

contd..

Native Asynchronous Handling Non-Blocking I/O: JavaScript natively supports async/await behaviors.

Perfect for Modern Web: Built to smoothly automate websites dependent on heavy API calls, WebSockets, or dynamic content loading.

4. **Massive Community and Package Support**

Reusable Packages: The npm registry provides thousands of ready-made libraries to scale your automation suite effortlessly.

TypeScript Evolution: Teams can scale to TypeScript to introduce strict typing, avoiding semantic and runtime mistakes before code execution even begins.

contd..

- ✓ **JavaScript** is premier language for **automation** and **scripting**, driven by its dominance in web technologies and the powerful server-side capabilities of Node.js.
- ✓ **JavaScript** allows developers and QA teams to bridge the gap between frontend user interfaces and backend processes seamlessly.
- 1. **Web Browser Automation** – JavaScript natively understands the Document Object Model (DOM), making it exceptionally efficient at controlling web browsers.
- ✓ **Headless Browser Control**: Operating browsers without a user interface to complete automated actions.
- ✓ **Dynamic Web Scraping**: Extracting data from modern, JavaScript-heavy sites that load content dynamically.
- ✓ **Form Auto-Filling**: Automating repetitive data entry tasks, checkout pipelines, or account creation.
- ✓ **PDF & Screenshot Generation**: Programmatically rendering web pages to generate reports or visual logs.

contd..

- ✓ **Key Frameworks:** Use Puppeteer or Playwright for reliable browser manipulation.
- 2. Software Quality Assurance & Testing** – Because most modern web applications are written in JavaScript, writing automation tests in the same language optimizes the development workflow.
- ✓ **End-to-End (E2E) Testing:** Simulating real user journeys across browsers to catch UI bugs.
- ✓ **API Test Automation:** Verifying backend endpoints by sending requests and validating payloads.
- ✓ **Visual Regression Testing:** Capturing screenshots of components to catch unintended layout shifts.
- ✓ **Mobile & Desktop Testing:** Simulating user interaction on native apps using cross-platform tools.
- ✓ **Key Frameworks:** Rely on Cypress, Playwright, or WebdriverIO.

contd..

3. Local File System & System Operations – Using Node.js, JavaScript functions as a robust shell scripting tool to manage local workstations or servers.

- ✓ **File Organization:** Monitoring folders like "Downloads" to sort files by extension automatically.
- ✓ **Automated Backups:** Copying critical assets and configuration directories to external storage.
- ✓ **Log Parsing:** Scanning heavy server logs to flag, filter, or extract error codes.
- ✓ **Bulk Renaming:** Batch-updating hundreds of filenames based on specific naming criteria.

4. **DevOps & CI/CD Pipelines** – JavaScript plays a major role in stitching together cloud infrastructure, build processes, and deployment routines.
- ✓ **Build Tools:** Orchestrating code compilation, minification, and asset bundling.
 - ✓ **Git Hooks:** Running scripts automatically before allowing code commits using Husky.
 - ✓ **Cloud Infrastructure:** Managing infrastructure-as-code deployment scripts using Pulumi.
 - ✓ **Deployment Triggers:** Sending status alerts to platforms like Slack when a deployment succeeds or fails.

contd..

- 5. Workspace Productivity & Chatbots** – JavaScript powers custom workflow extensions inside everyday workplace collaboration tools.
- ✓ **Google Workspace Automation:** Writing custom macros and data syncs via Google Apps Script.
 - ✓ **Slack & Discord Bots:** Creating automated conversational agents to manage support tickets or schedules.
 - ✓ **Notion & Jira Automation:** Leveraging JavaScript-based APIs to sync boards, create issues, or generate reminders.

Thank
You